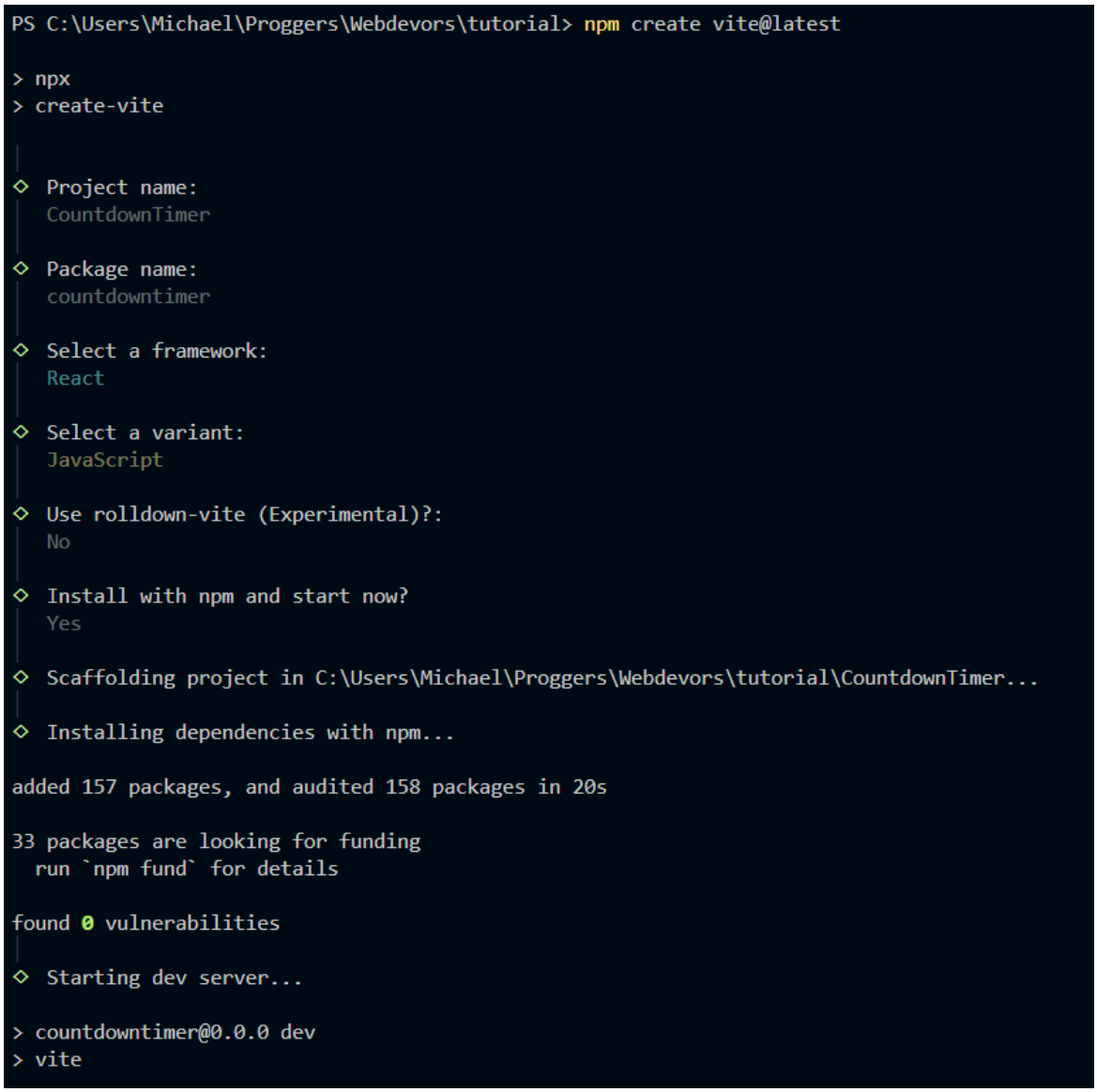
Get Started with ReactJS

1. Install Prerequisites

- Install Node.js LTS from nodejs.org. Verify both Node and npm:
  node -v
  npm -v
- (*Optional)* Install a package manager you prefer (npm ships with Node, so nothing else is required).

2. Scaffold a React Project with Vite

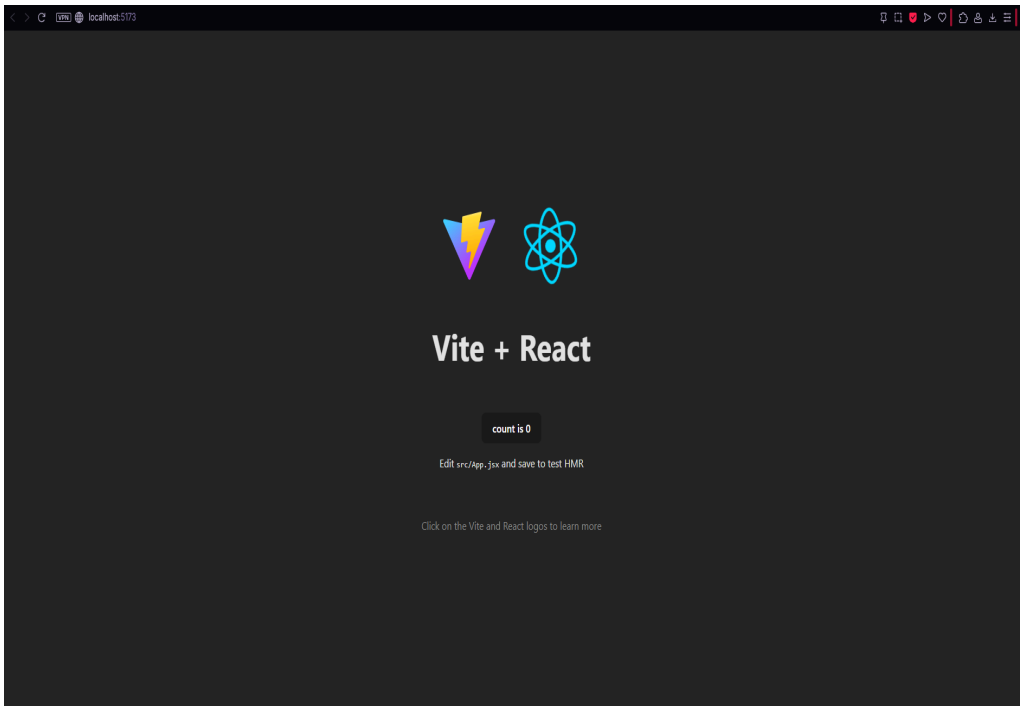
- Choose a file location and open it with your Windows Terminal
- Create a new Vite React project using this command on your terminal:
  npm create vite@latest
- You will be prompted to enter a Project name and Package name.
- Select *React* as framework and *Javascript* as variant



- Start the dev server:
  npm run dev

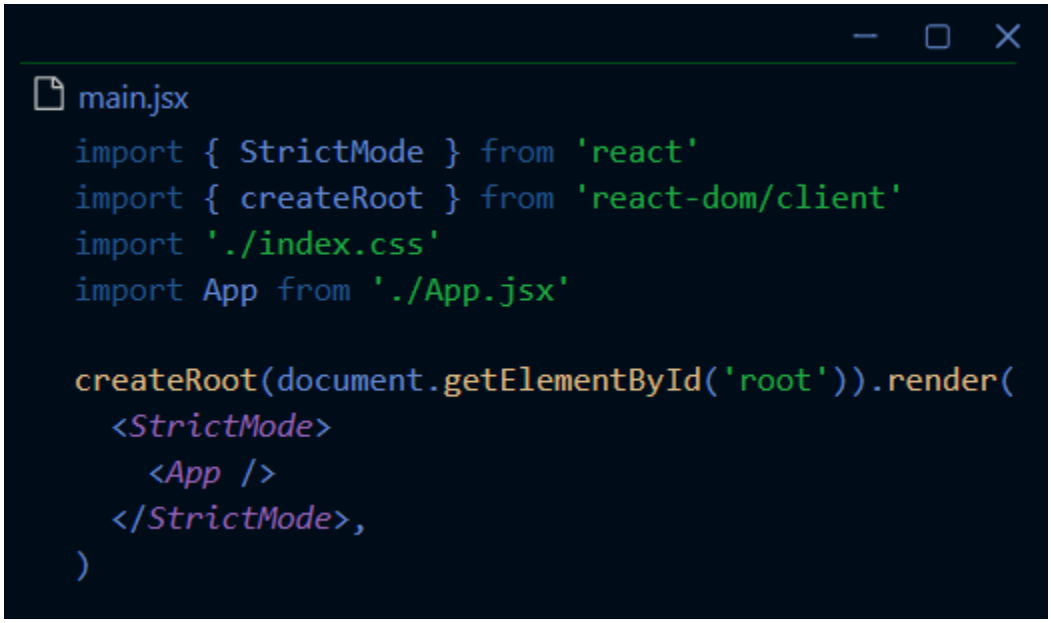
```
◇  Starting dev server...

> countdowntimer@0.0.0 dev
> vite


  VITE v7.2.2  ready in 345 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

- Vite prints a local URL (usually http://localhost:5173)—open it in the browser.



3. Understand the Entry Point

- Vite's React entry point renders the App component into the #root element. In this project it lives in *src/main.jsx:*



- When you export a component from App.jsx, Vite re-renders it automatically (thanks to Hot Module Replacement).

4. Creating Your First Component

- React components are just functions that return JSX. The TimeField component in App.jsx is a reusable input that enforces numeric typing:

App.jsx > TimeField

```jsx
const TimeField = ({ label, value, onChange, max }) => {
  const handleChange = (event) => {
    const next = event.target.value.replace(/\D/g, '').slice(0, 2)
    const numeric = next === '' ? '' : Math.min(parseInt(next, 10), max)
    onChange(numeric === '' ? '' : numeric.toString().padStart(2, '0'))
  }

  return (
    <label className="time-field">
      <span>{label}</span>
      <input
        type="text"
        inputMode="numeric"
        value={value}
        onChange={handleChange}
        placeholder="00"
      />
    </label>
  )
}
```

Try it yourself: *replace the JSX inside return with anything you like, and Vite instantly refreshes the browser so you can iterate quickly.*

5. Manage State with Hooks

- React's useState keeps track of changing data. The countdown stores hours, minutes, seconds, and the timer's current stage:

App.jsx > App

```jsx
function App() {
  const [hours, setHours] = useState('')
  const [minutes, setMinutes] = useState('')
  const [seconds, setSeconds] = useState('')
  const [status, setStatus] = useState('idle')
  const [remaining, setRemaining] = useState(0)
  const [duration, setDuration] = useState(0)
  const [stage, setStage] = useState('setup')
  const [initialLabel, setInitialLabel] = useState('00:00:00')
  const [isModalOpen, setIsModalOpen] = useState(false)
  const audioRef = useRef(null)

  const totalSeconds = useMemo(() => {
    const h = parseInt(hours || '0', 10)
    const m = parseInt(minutes || '0', 10)
    const s = parseInt(seconds || '0', 10)
    return h * 3600 + m * 60 + s
  }, [hours, minutes, seconds])

  const stopAudio = useCallback(() => {
    if (audioRef.current) {
      audioRef.current.pause()
      audioRef.current.currentTime = 0
    }
  }, [])
```

- **useState** returns the current value and an updater function.
- **useMemo** caches derived values (total seconds) so they're recalculated only when inputs change.
- **useCallback** memoizes handlers, keeping props stable for child components.

7. Compose the UI

- Inside the return block, React conditionally renders different layouts for setup vs. active timer stages:

App.jsx > App

```
return (
  <div className="page">
    <div className={`card ${stage === 'active' ? 'card--active' : ''}`}>
      <img src={timerIcon} className="title-image" alt="Warm up" />
      {stage === 'setup' ? (
        <>
          <p className="subtitle">Set your custom countdown</p>
          <div className="input-row">
            <TimeField label="Hours" value={hours} onChange={setHours} max={23} />
            <TimeField
              label="Minutes"
              value={minutes}
              onChange={setMinutes}
              max={59}
            />
            <TimeField
              label="Seconds"
              value={seconds}
              onChange={setSeconds}
              max={59}
            />
          </div>
          <button className="primary" onClick={handleBegin} disabled={!totalSeconds}>
            Start Timer
          </button>
        </>
      ) : (
        <>
          <button className="back-btn" onClick={handleBackToSetup} aria-label="Back to setup">
            Back
          </button>
          <CircularTimer duration={duration} remaining={remaining} status={status} />
          <div className="controls">
            <button
              className="control-btn play-btn"
              onClick={canToggle ? handleToggle : undefined}
              disabled={!canToggle}
              aria-label={playLabel}
            >
              {playGlyph}
            </button>
            <button className="control-btn reset-btn" onClick={handleReset} aria-label="Replay timer">

↺           </button>
          </div>
        </>
      )}
    </div>
    {isModalOpen && (
      <div className="modal-overlay">
        <div className="modal-card">
          <p className="modal-title">Timer Done</p>
          <p className="modal-time">{initialLabel}</p>
          <button className="primary1" onClick={handleDismiss}>
            Dismiss
          </button>
        </div>
      </div>
    )}
  </div>
)
```

This demonstrates:

- JSX expressions in { }.
- Conditional rendering (stage === 'setup' ? ... : ...).
- Passing props (duration, remaining, status) into child components.

And finally, add a styling using CSS or CSS frameworks (eg.: Tailwind, Bootstrap, Bulma). You can keep CSS modular by colocating *.css* files with components, or you can adopt CSS Modules, Tailwind, etc., but plain CSS works great for beginners.

Now you can try creating your own reactJS project of your choice.

Project source code used on this tutorial: https://github.com/itzmaiku44/Timer-Web-app
Feel free to modify it in your own style and more features.